

Потоковый ответ и отмена

Основной план, промпт 1.3

Краткое содержание

Streaming response — это не постепенно растущий `ModelResponse`, а упорядоченная последовательность предварительных `ModelEvent` и один `terminal outcome`. `TextDelta` можно показать как `provisional preview`, но нельзя фиксировать как завершённый ход Agent; частичные аргументы `tool` нельзя исполнять. `Cancellation`, `deadline` и `transport interruption` завершают ожидание по разным причинам и требуют разных решений runtime. `Retry` всегда создаёт новую модельную попытку, поэтому он расходует новый `budget` и может изменить текст или `tool call`.

Ключевые понятия

- **Observed prefix** — точная последовательность уже полученных `events`. Она полезна для UI, аудита и диагностики, но не является `terminal response`.
- **Terminal event** — событие, после которого результат больше не меняется. В текущем общем контракте это `ResponseCompleted`.
- **Provisional state** — обратимое представление незавершённого ответа, например `streaming preview` в UI. Это не `committed Agent state`.
- **Interruption** — `stream` закончился без `terminal event` из-за EOF, `transport failure` или другой ошибки чтения.
- **Caller cancellation** — вызывающая сторона больше не хочет продолжать операцию. В `asyncio` это кооперативный сигнал `CancelledError`.
- **Deadline** — абсолютный момент окончания общего временного `budget`, а не новый локальный `timeout` на каждом слое.
- **Backpressure** — ограничение скорости чтения скоростью `downstream consumer`, чтобы очередь `events` и память не росли без границ.
- **Retry budget** — общий предел `attempts`, времени и стоимости. Он важнее желания отдельного `adapter` «попробовать ещё раз».

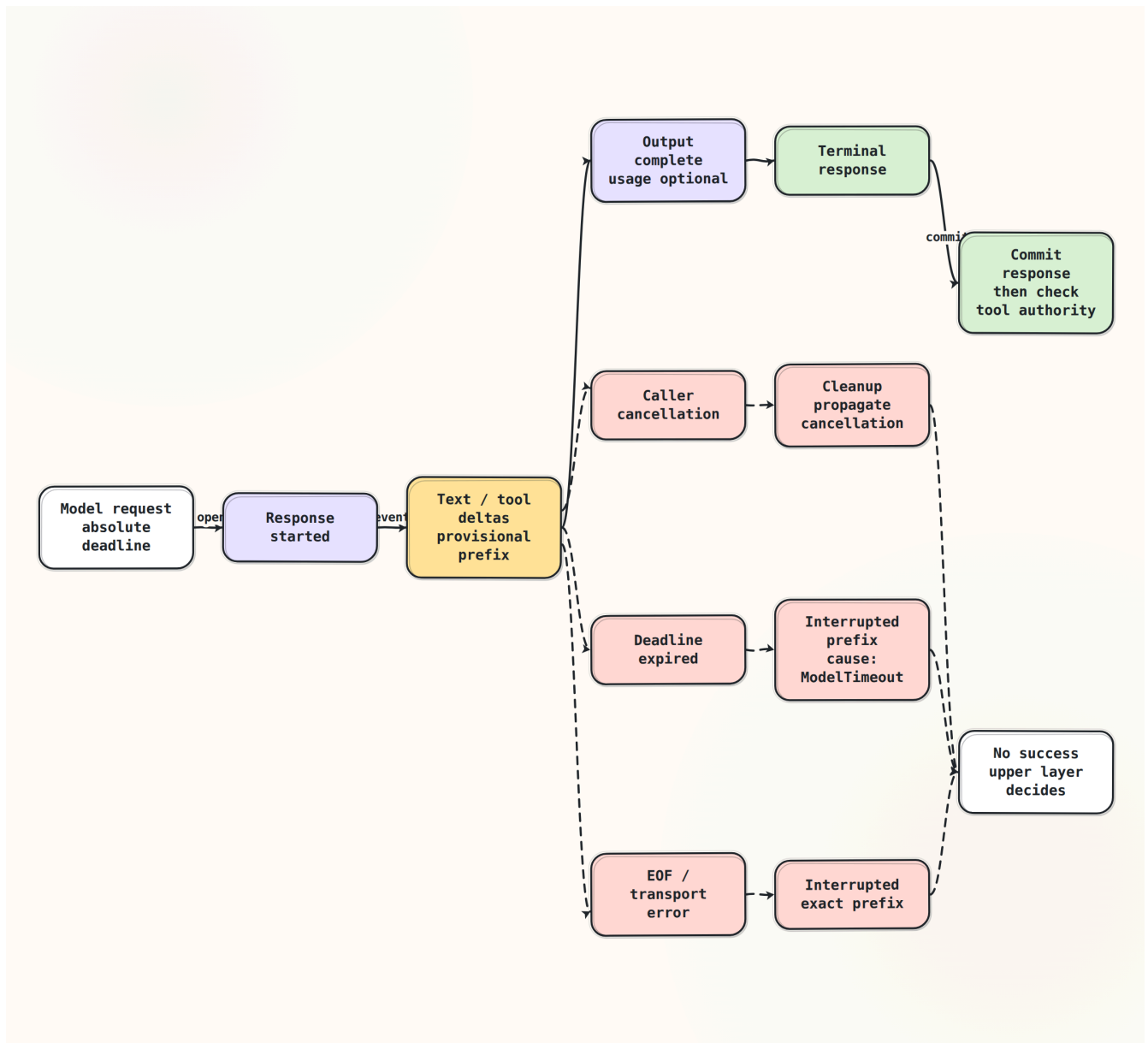
Основные идеи

Один stream, две шкалы состояния

Transport может уже передать десятки `chunks`, пока `semantic response` всё ещё не завершён. Поэтому runtime ведёт два разных представления:

1. `observed_events` растёт после каждого проверенного `event`;
2. `committed_response` остаётся пустым до `ResponseCompleted`.

Это различие защищает Agent от преждевременного решения. Видимый текст можно отменить или пометить незавершённым. Исполненный `tool` уже мог изменить внешнюю систему, поэтому `partial ToolArgumentsDelta` остаётся только строкой до `OutputCompleted`, `terminal event` и проверки полномочий.



Главный инвариант: ни EOF, ни cancellation, ни deadline не создают синтетический `ResponseCompleted`. Только настоящий `terminal event` разрешает перенести response из provisional state в committed Agent state.

Жизненный цикл успешного ответа

1. Runtime формирует `ModelRequest`, `attempt identity` и один абсолютный `deadline`.
2. Adapter открывает stream; `ResponseStarted` связывает его с request и provider response.
3. `TextDelta` и `ToolArgumentsDelta` расширяют `observed prefix`. Event order, provider identity и output index проверяются сразу.
4. `OutputCompleted` доказывает целостность конкретного output. Для streamed JSON собранные chunks должны совпасть с полным raw value.

5. `UsageReported` может появиться поздно либо отсутствовать.
6. `ResponseCompleted` сверяет `request/provider identity`, `completed outputs` и `reported usage`. Сам `ModelResponse` дополнительно проверяет допустимое сочетание `outcome` и `output variants`. Только после этого runtime получает `terminal result` для следующего шага Agent.

`ResponseOutcome.TRUNCATED` тоже `terminal`: provider закончил генерацию из-за лимита. Это неполный результат LLM, но не `transport exception`. Решение увеличить `token limit`, запросить `continuation` или выбрать другую модель принадлежит Agent runtime и создаёт новый `exchange`.

Cancellation и deadline — не одно и то же

Оба механизма могут технически отменить ожидающую `coroutine`, но выражают разный intent:

- внешний `CancelledError` означает «caller больше не ждёт»; runtime отменяет `active read` и `deadline waiter`, закрывает `source`, после чего `cleanup adapter generator` выполняется в `finally`. Cancellation проходит наверх без `hidden retry`;
- истёкший `deadline` до первого event либо во время `retry wait` даёт `ModelTimeout` с конкретной `phase`;
- истёкший `deadline` после `provisional events`, но до `validated ResponseCompleted`, даёт `ModelStreamInterrupted(events, cause=ModelTimeout)`, а не `completed response`.

Deadline должен передаваться как абсолютное значение, измеренное `monotonic clock`. Иначе `transport`, чтение `stream` и каждый `retry` получают полный `timeout` заново, а реальная операция превышает обещанный пользователю `budget`.

В Python 3.13 `asyncio.timeout_at(deadline)` использует абсолютное время `event loop`. Контекст применяет `cancellation` внутри и преобразует именно своё истечение в `TimeoutError` снаружи. До `terminal commit` обычный `CancelledError` нельзя поглощать, иначе ломаются `cleanup` и `structured concurrency`. После `validated ResponseCompleted` текущая `policy` делает узкое исключение: завершает `cleanup`, возвращает `committed response` и сохраняет `cancellation` как `cleanup_error`.

429 и Retry-After

HTTP 429 Too Many Requests сообщает о `rate limiting`. `Retry-After` — `server hint` о том, как долго следует подождать; это не гарантия успеха и не разрешение игнорировать общий `budget`.

Runtime может повторить `exchange`, только если одновременно выполнены условия:

- `retry` разрешён политикой для этой фазы;
- попытки и ожидаемая стоимость укладываются в `retry budget`;
- ожидание вместе с новой попыткой помещается в оставшийся `deadline`;
- `sleep` прерывается `caller cancellation`;
- предыдущая попытка не создала `terminal Agent transition` или внешний `side effect`.

Adapter должен вернуть `typed ModelRateLimited(retry_after_s=...)`, а решение об ожидании оставить `orchestration layer`. При отсутствующем или `invalid Retry-After` `production runtime`

обычно применяет bounded backoff с jitter. В детерминированной практике используется явная fallback delay без jitter; если server hint или fallback delay не помещаются в оставшийся deadline, корректный исход — timeout, а не ожидание сверх budget.

Что означает безопасный retry

Здесь «безопасный» означает прежде всего отсутствие повторного внешнего side effect. Даже чистая генерация LLM при одинаковом request может дать другой ответ, другую стоимость и другую latency.

Наблюдение	Решение runtime
429 до первого event	Bounded retry допустим после Retry-After , если хватает deadline и budgets
Deadline до первого event	Внутри этой операции retry невозможен: budget исчерпан. Только новый upper-level exchange с новым deadline и policy
Обрыв после text delta	Сохранить/пометить prefix; новый stream не дописывать к старому
Обрыв внутри tool arguments	Не парсить и не исполнять partial JSON; новый attempt должен начать proposal заново
Caller cancellation	Cleanup и CancelledError наверх; автоматический retry запрещён
Terminal TRUNCATED	Не transport retry; отдельное решение о continuation, fallback или stop
usage=None	Не retry; стоимость остаётся неизвестной, а не нулевой

Если preview уже показан пользователю, новая попытка должна заменить его или начать отдельный ответ с явной маркировкой. Молчаливое склеивание old prefix и new stream создаёт текст, которого не выдавал ни один model exchange.

Причинная связь с Agent

- TextDelta → меняется только provisional UI → tool недоступен → обрыв не создаёт внешнего эффекта.
- OutputCompleted → output целостен, но весь response ещё не terminal → runtime продолжает ждать → параллельные outputs и usage не теряются.
- ResponseCompleted → появляется committed ModelResponse → Agent может принять следующее решение или передать завершённый tool call на отдельные schema, allowlist и authorization checks перед executor.
- Cancellation/deadline/interruption до validated ResponseCompleted → committed response отсутствует → retry решает runtime по общей policy → adapter не скрывает новый model decision.

Примеры

Provisional preview без преждевременного commit

```
async def preview(source):
    async for event in source:
        audit.append(event)
        if isinstance(event, TextDelta):
            ui.preview(event.delta) # reversible, not Agent state
        yield event

collected = await collect_stream(preview(client.stream(request)))
state.commit(collected.response) # only after lifecycle validation
```

Если после `TextDelta("Возьмите зон")` соединение оборвалось, UI может показать незавершённый prefix, но history Agent не получает обычный assistant message. Если terminal response имеет `outcome=TRUNCATED` и `usage=None`, state честно сохраняет оба факта: ответ неполон, стоимость неизвестна.

Один абсолютный deadline

В практике эту границу реализует `StreamingRuntime`: один объект `Deadline` передаётся каждой видимой попытке, ограничивает чтение stream и ожидание перед retry.

```
deadline = Deadline(clock.now() + 10)
result = await StreamingRuntime(time=clock).collect(
    client,
    request,
    deadline=deadline,
)
```

Сам `Deadline` относится к одной monotonic time domain. Передавать его как обычный timestamp между процессами нельзя: после durable restart нужно сохранять переносимое wall-clock значение или оставшийся budget и строить новый local monotonic deadline.

Заметки и наблюдения

Что доказано практикой 1.3

- `StreamingRuntime` создаёт `ModelAttempt`, передаёт один absolute `Deadline` каждой попытке и ограничивает им чтение stream и retry wait.
- Caller cancellation доходит до заблокированного source как `CancelledError`: source закрывается, deadline waiter удаляется, новая попытка не открывается.
- Transport failure или deadline после принятых events возвращаются как `ModelStreamInterrupted` с точными event objects и исходной причиной. Ни

`ModelResponse`, ни синтетический `terminal event` не создаются.

- `Bare ModelRateLimited` до первого event допускает `bounded retry`. После первого event та же ошибка становится причиной `ModelStreamInterrupted`, поэтому runtime не склеивает две генерации.
- `ResponseOutcome.TRUNCATED` остаётся `terminal response`, а отсутствующий `usage` сохраняется как `None`.
- После `validated ResponseCompleted` ошибка или `cancellation` во время `cleanup` записывается в `cleanup_error` и не отменяет `committed response`.

Две наблюдаемые трассы находятся в `demo.py`, а `fault matrix` — в `test_execution.py`.

Итог инженерной защиты: `tool authority`

Сценарий защиты использует syntactically complete `ToolArgumentsDelta('{"sku": "sku-42", "quantity": 2}')`, после которого `transport` обрывается до `OutputCompleted` и `ResponseCompleted`. Выбранная `policy`:

1. Agent остаётся в состоянии ожидания подтверждённого решения; `prefix` — это `draft`, а не `ToolCallOutput`.
2. Runtime возвращает `ModelStreamInterrupted` без `hidden retry`.
3. Application может предложить отмену или отдельную новую генерацию после решения пользователя. «Подтвердить вручную» не означает исполнить JSON из `prefix`: отдельная команда должна снова пройти `schema validation` и проверку полномочий.
4. Только `ToolCallOutput` из одного `validated ResponseCompleted` может стать кандидатом на `authority` для `reserve_inventory`; затем обязательны `schema`, `allowlist` и `authorization checks`. Events разных `attempts` не объединяются.

Falsification test

`test_complete_tool_arguments_interruption_never_retries_or_authORIZES_tool` проверяет отрицательный и положительный `control`. В `interrupted branch` полный JSON сохраняется как `event`, `attempt` остаётся один, а `spy tool` не вызывается. В отдельном новом `request` последовательность `OutputCompleted` → `ResponseCompleted` делает `call eligible` для `test policy`, которая разрешает ровно один вызов `spy tool`. Это доказывает `terminal boundary`, но не заменяет полную `production authorization policy`.

Ограничения текущего доказательства

- `ControlledStreamingClient` и `ManualTime` доказывают локальный `control flow`, но не `remote cancellation`, остановку `provider compute` или отсутствие `billing` после закрытия соединения.
- Здесь нет реального HTTP/SSE adapter, `durable checkpoint/replay` после `process crash` и отдельного `cost budget`.
- `Fallback delay` детерминирована и не моделирует `jitter` либо `contention` между несколькими `workers`.
- `collect_stream` сохраняет все `events` и `chunks` в памяти; `bounded buffering` и `end-to-end backpressure` не измерены.
- `Caller cancellation` проходит наружу без `prefix`. Для `audit trail events` нужно писать в отдельный `sink` до `terminal commit`.

- Общего resume protocol нет. Новый request начинает новый model exchange, а correlation ID сам по себе не даёт idempotency.

Гонки на terminal boundary

Тесты фиксируют две стороны границы. Если deadline и ещё не принятый terminal event становятся ready в одном event-loop turn, побеждает deadline и terminal response не синтезируется. Если `ResponseCompleted` уже validated, а deadline или cancellation приходят во время cleanup, побеждает committed response; проблема cleanup остаётся наблюдаемой через `cleanup_error`.

Вопросы для повторения

1. Почему полученный text prefix ещё не является `ModelResponse`?
2. Как partial `ToolArgumentsDelta` может превратить transport bug в опасный side effect?
3. Почему внешний cancellation нельзя автоматически преобразовать в timeout?
4. Зачем передавать один absolute deadline через transport и retry wait?
5. Когда 429 допускает retry и почему `Retry-After` недостаточно само по себе?
6. Почему `TRUNCATED` — terminal result, а не transport failure?
7. Что обязан сохранить runtime, если usage отсутствует?
8. Почему новый stream нельзя продолжить с последнего text delta без явной поддержки resume protocol?

Итоги

Streaming уменьшает time to first visible token, но усложняет границу commit. Observed prefix принадлежит provisional state; только validated terminal event создаёт `ModelResponse`.

Cancellation сохраняет intent caller, deadline ограничивает всю операцию, а interruption сохраняет неоднозначный prefix. Retry остаётся явным решением runtime с общими time/attempt budgets и не маскируется под продолжение потока. Отдельный cost budget в практике не реализован и остаётся обязанностью upper layer.

Практика завершена без framework: реализованы deadline-aware stream runtime, deterministic clock и fault cases для interruption, cancellation, 429, token limit, missing usage и terminal cleanup. Карта запуска и ограничения собраны в `README`.

Источники

Изменчивые детали Python и HTTP semantics проверены по primary sources 2026-08-02; локальная практика повторно запущена 2026-08-12 на Python 3.13.13:

- Python 3.13: task cancellation
- Python 3.13: timeouts and `asyncio.timeout_at`
- RFC 6585: 429 Too Many Requests
- RFC 9110: `Retry-After`
- Основной план: промпт 1.3

- Локальные материалы: contracts, stream assembler, runtime, tests, README и конспект 1.2